RL for Diffusion

Andrei Filatov · 2026
t.me/awesome_dl · anvilarth.github.io

Preference Optimization

Pretraining is about {modality1}-{modality2} alignment. But it's lacking visual prioritization (e.g. this lighting is better, this texture is worse, etc.)





DPO Recap

- DPO is all about data – Higher margins easier training
- Main feature is off-policy: easy to collect data, but easy to fall into saturation. (even cfg creates biases but you can do cfg distillation implicitly)

Preference Optimization

What type of preference optimization do we have?

1. **Pair-based – DPO**. Explicit feedback model/person – this one is better / this one is worse

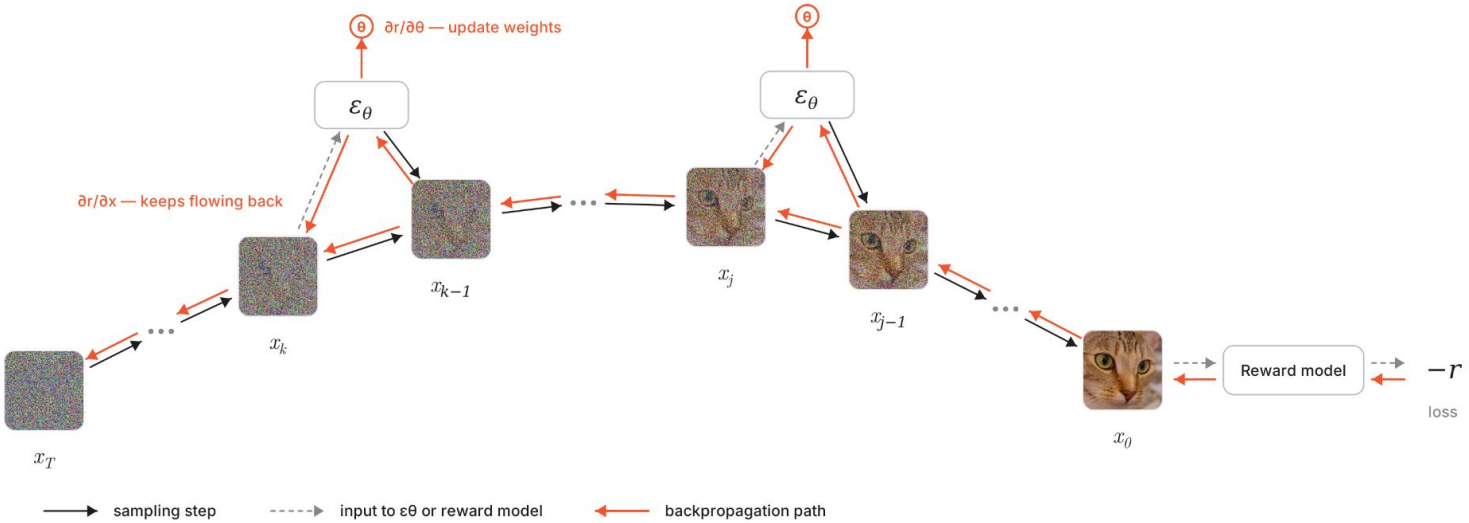
2. **Reward based – RL, GRPO**, etc. Reward model provides granular feedback (OCR, Aesthetics)

RL problems

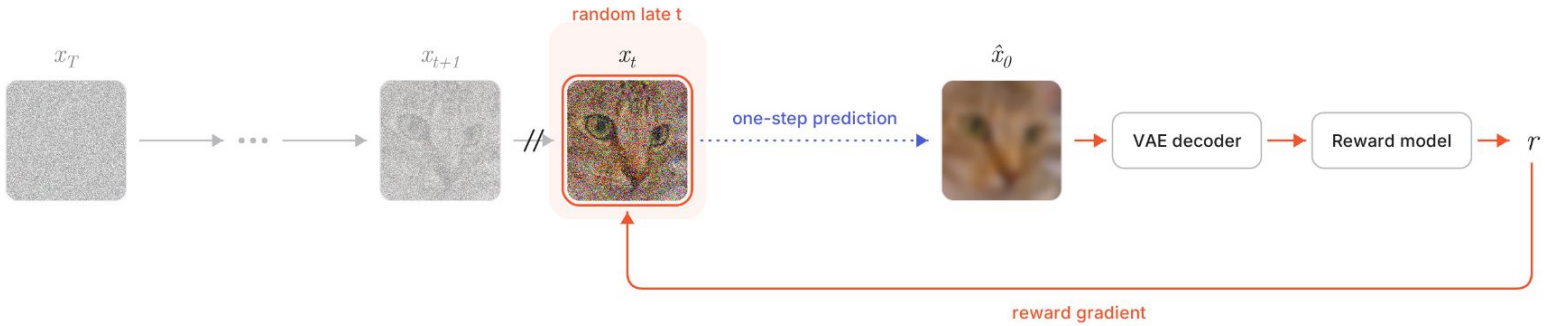
1. Signal propagation:
   a. Direct reward backpropagation
   b. Online policy gradient
   ~~c. DPO (prev. lecture)~~

2. Reward model construction (next lecture)
   a. How to calculate reward?
   b. How to balance it?

Direct reward backpropagation

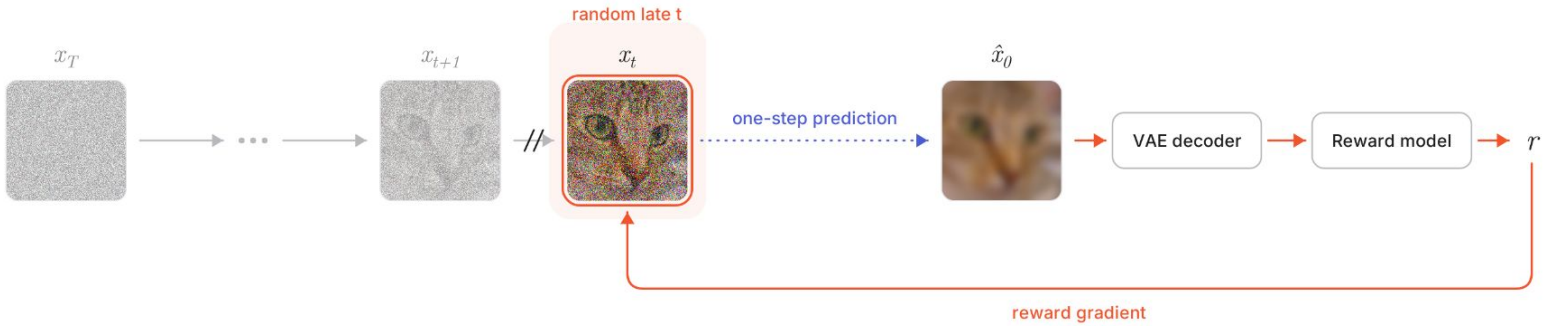


ReFL



Backpropagate the reward through one random step in the middle of the denoising trajectory

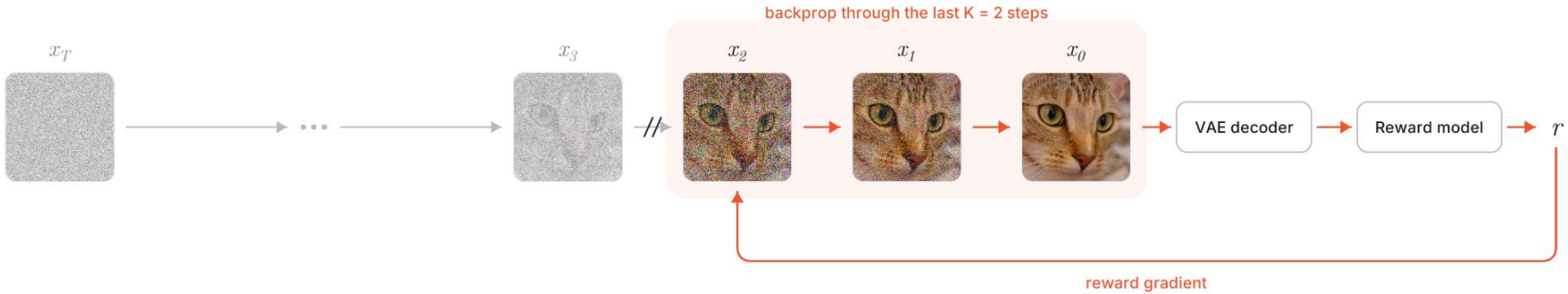
ReFL



How to pick random step? Just pick something in the middle

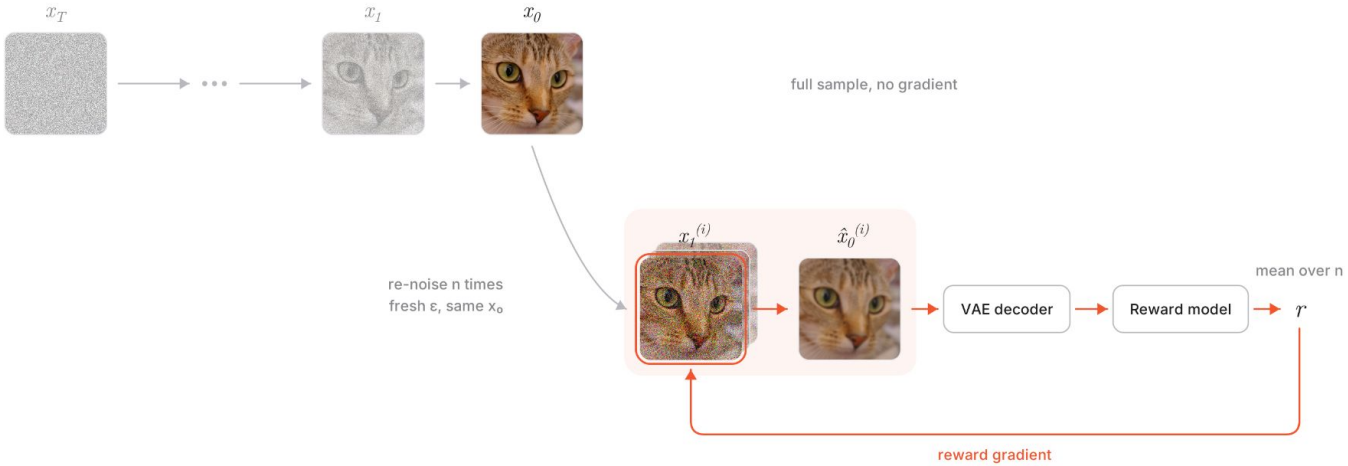
- Early steps: too noisy
- Late steps: hard to update

DRaFT-K



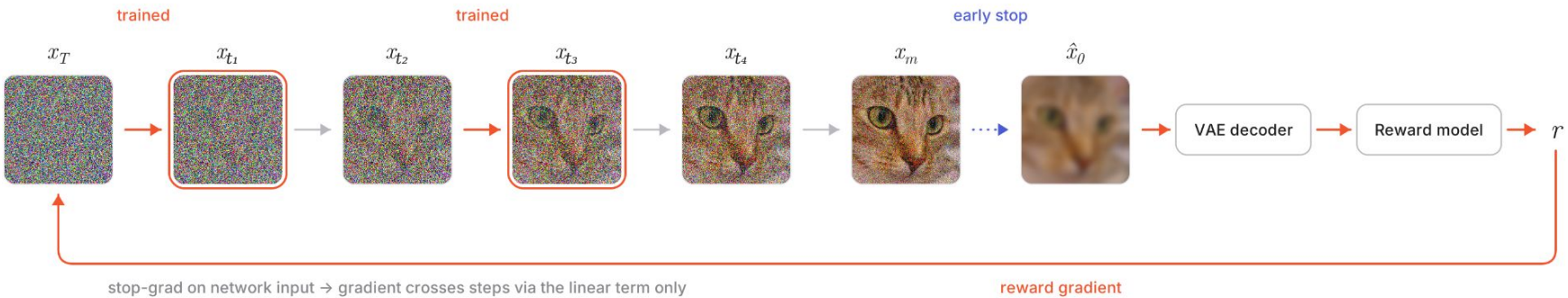
Backpropagate the reward through the last K sampling steps, not just one

DRaFT-LV



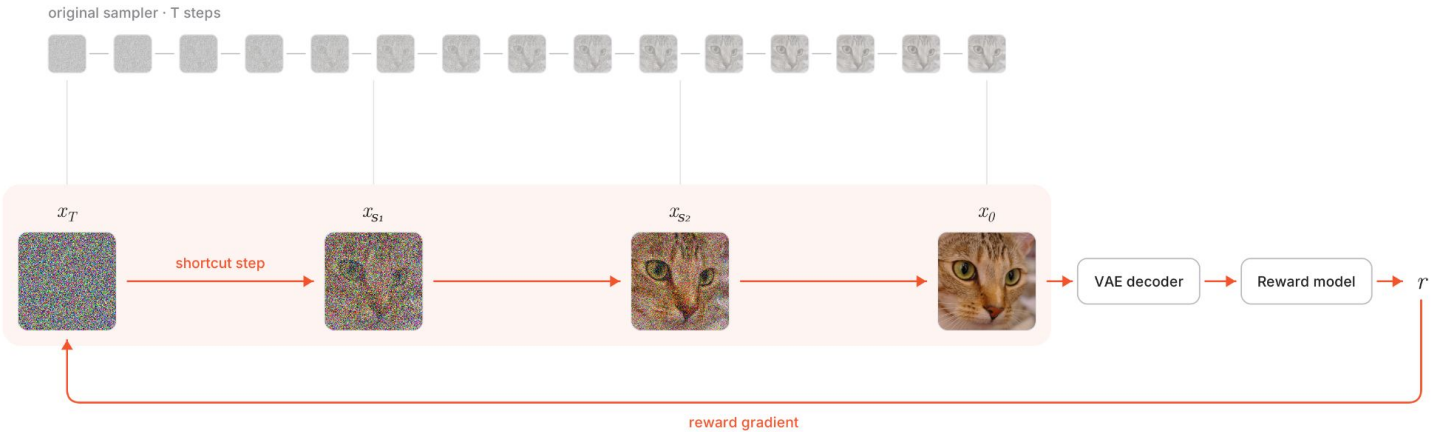
Denoise x_0 multiple times to reduce variance

DRTune



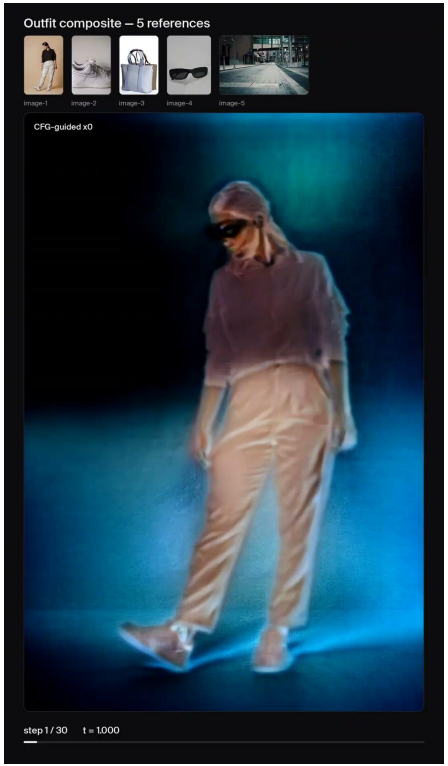
How to update early steps to improve structure without gradient blow-up?

ShortFT



What if we update the full model through a shortcut (few-step) model?

Trajectory example



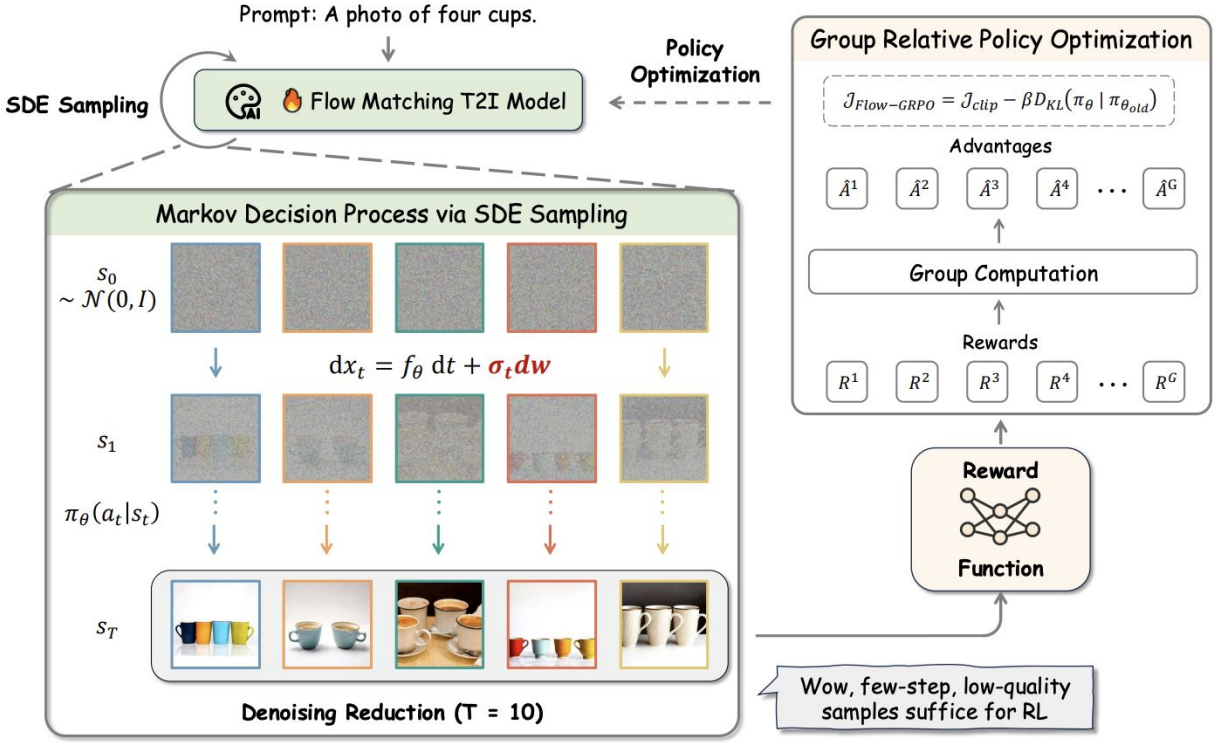
Online policy gradient

1. Reward is non-differentiable (a lot of oracles are binary)
2. Gradient of RM is sus
3. ONLINE!!!!

$$s_t = (x_t, t, c), \qquad a_t = x_{t-1}, \qquad \pi_\theta(a_t \mid s_t) = p_\theta(x_{t-1} \mid x_t, c), \qquad r = R(x_0, c) \text{ at the end}$$

$$\nabla_\theta J(\theta) = \mathbb{E}_{c,\tau}\Big[\big(R(x_0, c) - b(c)\big) \sum_{t=1}^{T} \nabla_\theta \log \pi_\theta(x_{t-1} \mid x_t, c)\Big]$$

Flow-GRPO

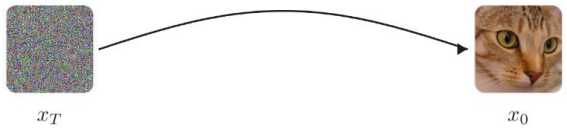

Prompt: A photo of four cups.
SDE Sampling
Flow Matching T2I Model
Policy Optimization
Group Relative Policy Optimization
$\mathcal{J}_{Flow-GRPO} = \mathcal{J}_{clip} - \beta D_{KL}(\pi_\theta \mid \pi_{\theta_{old}})$
Advantages
$\hat{A}^1$ $\hat{A}^2$ $\hat{A}^3$ $\hat{A}^4$ ... $\hat{A}^G$
Group Computation
Rewards
R^1 R^2 R^3 R^4 ... R^G
Reward Function
Markov Decision Process via SDE Sampling
$s_0 \sim \mathcal{N}(0, I)$
$dx_t = f_\theta\, dt + \sigma_t dw$
s_1
$\pi_\theta(a_t|s_t)$
s_T
Denoising Reduction (T = 10)
Wow, few-step, low-quality samples suffice for RL

Flow-GRPO

How to add diversity?

ODE sampling

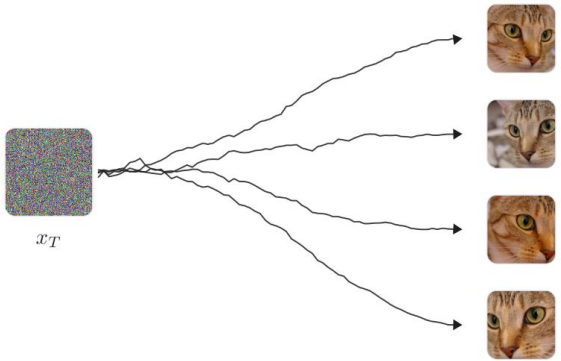
$$dx_t = v_\theta(x_t, t)\, dt$$



next step is fixed once x_t is fixed
no exploration · no log-probability

SDE sampling

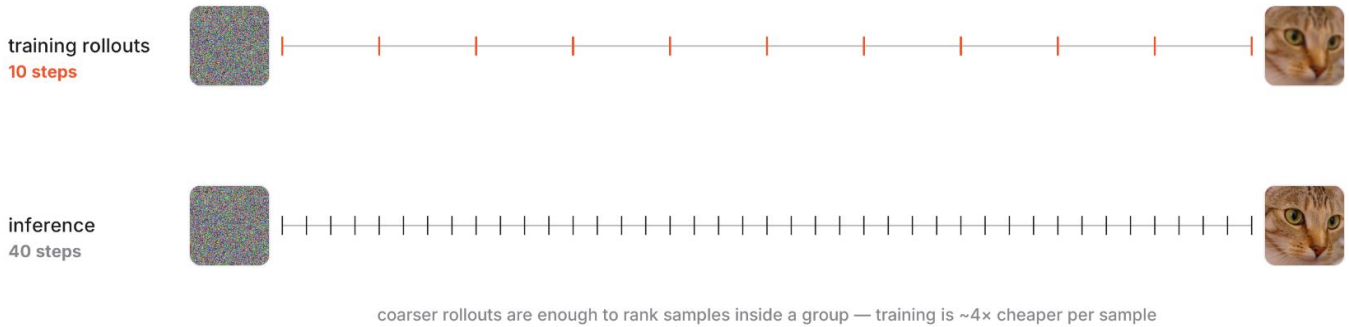
$$dx_t = \left[v_\theta + \frac{\sigma_t^2}{2t}\left(x_t + (1-t)\, v_\theta\right)\right] dt + \sigma_t\, dw$$



Gaussian steps: exploration + tractable log π

same marginals $p_t(x_t)$ as the ODE, $\sigma_t = a\sqrt{t/(1-t)}$

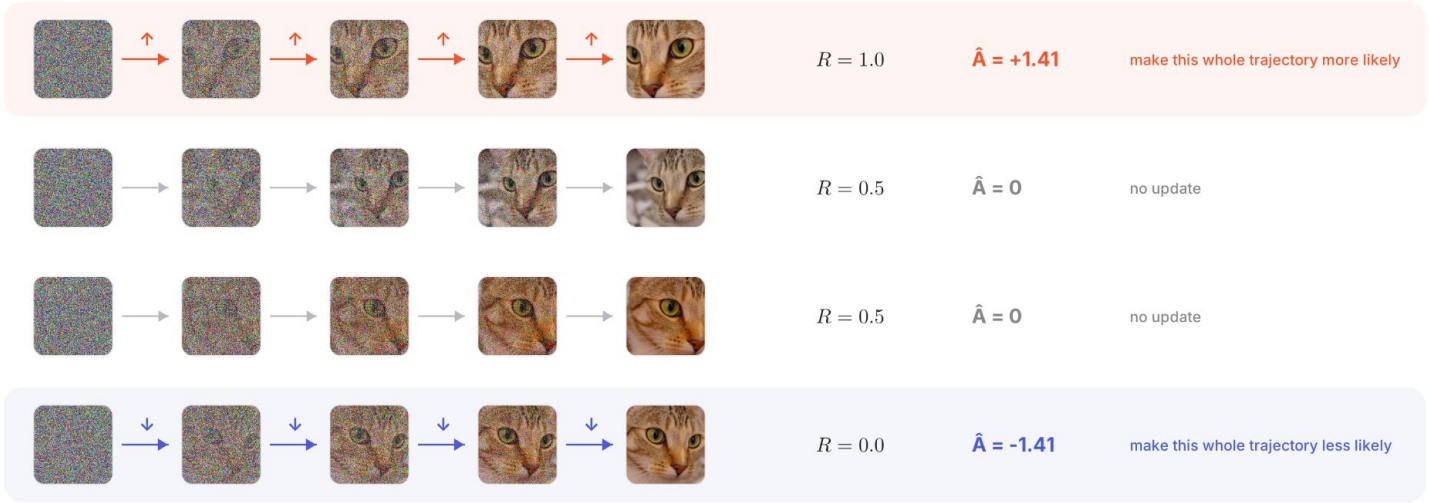
ShortFT works



Flow matching gives straighter trajectories, so we can train on fewer steps

Doing some GRPO stuff

$$\nabla_\theta J \approx \frac{1}{G}\sum_{i=1}^{G}\hat{A}_i \sum_{t=1}^{T}\nabla_\theta \log \pi_\theta\left(x_{t-1}^{(i)} \mid x_t^{(i)}, c\right)$$



every denoising step of a trajectory gets the same advantage — early steps (layout) included

DiffusionNFT

Problems of Flow-GRPO:

- Training kinda weird (we optimize backward process instead of forward)
- Depends on sampler (only Euler applicable)
- CFG training is necessary

Solution:

- Let’s do rejection finetuning on good stuff – meh, we need negatives

DiffusionNFT

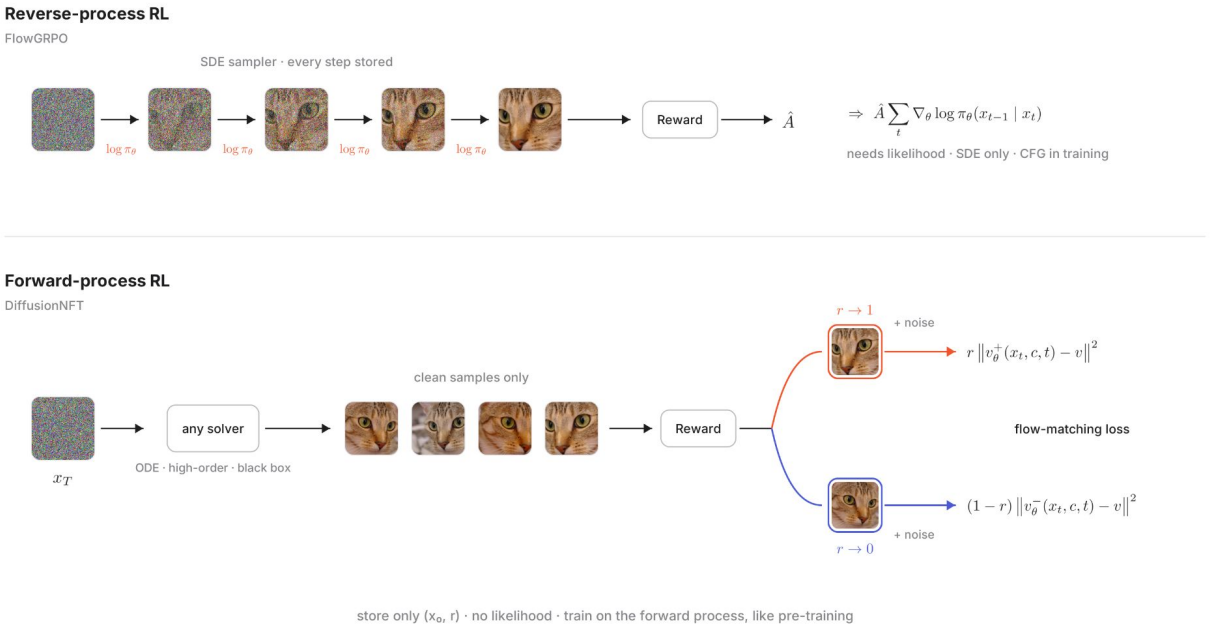
Problems of Flow-GRPO:

- Training kinda weird (we optimize backward process instead of forward)
- Depends on sampler (only Euler applicable)
- CFG training is necessary

Solution:

- Let’s do rejection finetuning on good stuff – meh, we need negatives

DiffusionNFT



DiffusionNFT

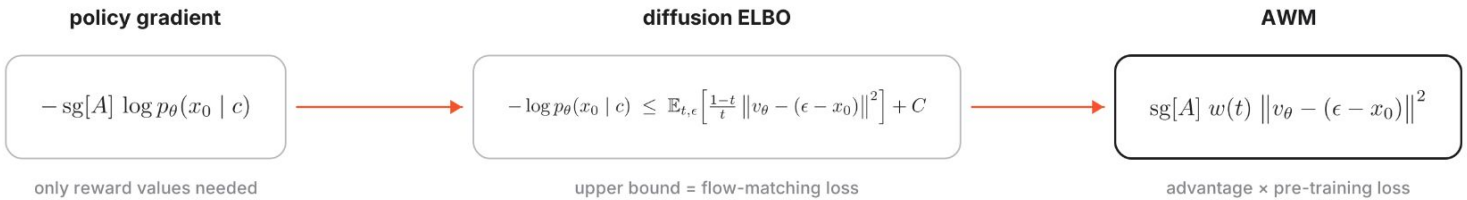
$$v_\theta = v_{\text{old}} + \delta \;\Rightarrow\; v_\theta^{\pm} = v_{\text{old}} \pm \beta\delta$$

$$\nabla_\delta \mathcal{L}\Big|_{\delta=0} = 2\beta \underbrace{(2r-1)}_{\in[-1,1]} \big(v_{\text{old}} - v\big) \quad \Rightarrow \quad \delta \;\propto\; (2r-1)\big(v - v_{\text{old}}\big)$$

Advantage Weighted Matching

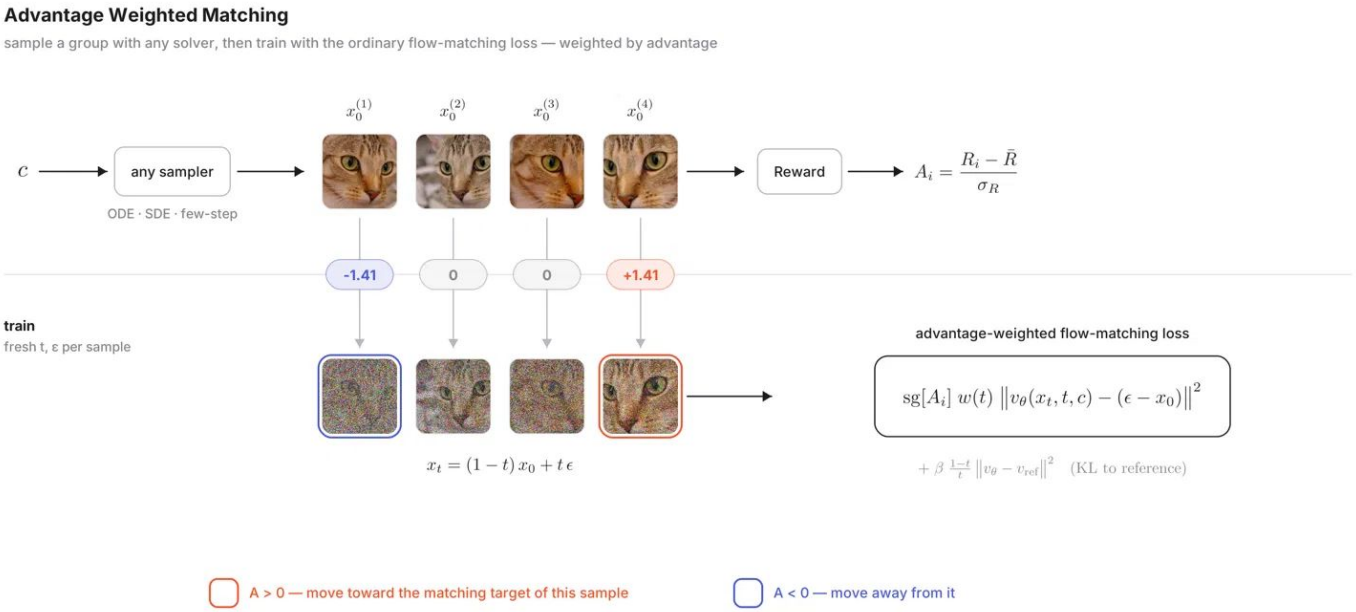
From policy gradient to a matching loss

replace the intractable log-likelihood with the diffusion ELBO — which is the pre-training loss



$A > 0:$ pull v_θ toward $(\epsilon - x_0)$ $\qquad$ $A < 0:$ push it away

Advantage Weighted Matching



Summary

1. Diffusion RL is equal to weighted flow matching
2. Propagate through whole trajectory
3. Data is the moat

References

Direct Preference Optimization (DPO) — https://arxiv.org/abs/2305.18290

Diffusion-DPO — https://arxiv.org/abs/2311.12908

ImageReward / ReFL — https://arxiv.org/abs/2304.05977

DRaFT (DRaFT-K, DRaFT-LV) — https://arxiv.org/abs/2309.17400

DRTune: Deep Reward Supervisions — https://arxiv.org/abs/2405.00760

ShortFT: Shortcut-based Fine-Tuning — https://arxiv.org/abs/2507.22604

Flow-GRPO — https://arxiv.org/abs/2505.05470

DiffusionNFT — https://arxiv.org/abs/2509.16117

Advantage Weighted Matching — https://arxiv.org/abs/2509.25050